

BEE 4750 Homework 2: Systems Modeling and Simulation

Name: Sarah Vafiadis, Nathan Rhoads

ID: sv439, nar84

Due Date

Thursday, 09/25/25, 9:00pm

Overview

Instructions

- Problem 1 asks you to draw a systems diagram and identify the type of a feedback.
- Problem 2 asks you to model contaminant concentrations in a river and use simulation to compare the concentrations to a regulatory standard.
- Problem 3 asks you to explore the implications of an ice-albedo feedback in the Earth's climate system by understanding the equilibria of the modeled system and their stabilities.

Load Environment

The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
import Pkg
Pkg.add("GraphPlot")
Pkg.activate(@__DIR__)
Pkg.instantiate()
```

Resolving package versions...

```

using Plots
using LaTeXStrings
using CSV
using DataFrames
using Roots
using GraphRecipes

```

Problems (Total: 30 Points)

Problem 1 (5 points)

Draw a systems diagram for the relationship between global mean temperature, atmospheric CO₂ concentrations, and ocean CO₂ concentrations. What are the signs of the interactions between these components and why? What does this suggest about the overall feedback between temperature and the ocean carbon cycle?

```

A = [0 1 0;
      1 0 1;
      0 0 0]

# Node names
nodes = ["Ocean CO ", "Atmospheric CO ", "Global Mean Temperature"]

# Edge labels
edgelabels = Dict{
    (1, 2) => "negative",
    (2, 1) => "positive",
    (2, 3) => "positive"
}

# Draw graph
graphplot(
    A,
    names = nodes,
    nodeshape = :rect,
    nodestrokecolor = :black,
    nodestrokewidth = 2,
    nodecolor = :lightblue,
    fontsize = 10,
    edgelabel = edgelabels,
    arrowsize = 0.8,

```

```

    arrow = true
)

```

The sign between the ocean to the atmosphere is negative, because the ocean absorbs CO₂ from the atmosphere. Since the atmospheric CO₂ concentration wants to be in equilibrium with the ocean (Henry's Law), human release of CO₂ into the atmosphere is generally absorbed by the ocean, decreasing the concentration in the atmosphere. This makes the arrow from the atmosphere to the ocean positive, since it is increasing the concentration in the ocean. The higher levels of CO₂ in the atmosphere drives the mean temperature up due to CO₂'s greenhouse gas effect.

Tip

Think about Henry's law for CO₂.

Problem 2 (15 points)

A river which flows at 10 km/d is receiving discharges of wastewater contaminated with CRUD from two sources which are 15 km apart, as shown in the Figure below. CRUD decays exponentially in the river at a rate of 0.36 d^{-1} and is deposited by the atmosphere along the river at a rate of 54 kg/km/d.

Schematic of the river system in Problem 2

Problem 2.1

Draw a systems diagram with the relevant control volume(s) denoted and any relevant in/out-flows between these boxes. How did you decide how many boxes were needed?

```

A = [0 0 0 1 0;
     0 0 0 1 0;
     0 0 0 1 0;
     0 0 0 0 0;
     0 0 0 1 0]

graphplot(
    A,
    names = ["Discharge", "Discharge", "Atmosphere", "River\n0.36 d^-1", "Inflow"],
    nodeshape = [:circle, :circle, :circle, :rect, :circle],
    edgelabel = Dict{(:3, 4)} => "54 kg/km/d",
    nodecolor = :white,
    markersize = 0.25,

```

```

arrow = true,
x = [-1.0, 1.0, 0.2, 0.0, -1.25],
y = [1.25, 1.25, 1.25, 0.0, 0.0],
xlims = (-2, 2.5),
ylims = (-0.5, 2)
)

```

I only included one control volume - the river. There are multiple flows within the river at different points, but the overall system is limited to the river itself. The circles represent inflows and outflows.

Problem 2.2

Develop a model for the concentration of CRUD downriver by formulating and solving the appropriate differential equation(s) analytically.

Tip

Formulate your model in terms of distance downriver, rather than leaving it in terms of time from discharge.

Differential equation defining change in concentration

$$\frac{dC}{dx} = \frac{a}{Q} - \frac{kC}{v}$$

using Plots

```

# Parameters
v = 10.0          # river velocity (km/d)
k = 0.36          # decay rate (d^-1)
a = 54.0          # atmospheric deposition (kg/km/d)
Q = 250000        # river flow rate (m3/d)
Q_d = 0.5 / 1000  # CRUD concentration in river inflow (kg/m3)
s1 = 40000        # discharge source 1 flow rate (m3/d)
s1_d = 9.0 / 1000 # CRUD concentration in discharge source 1 (kg/m3)
s2 = 60000        # discharge source 2 flow rate (m3/d)
s2_d = 7.0 / 1000 # CRUD concentration in discharge source 2 (kg/m3)

x1 = 0            # position of source 1 (km)
x2 = 15           # position of source 2 (km)

```

```

# Equilibrium concentration
function no_mix(Q_flow)
    atmosphere = a * v / (k * Q_flow)
    return atmosphere
end

# Mixing at each stopping point
function mix(Q_i, C_i, dis_flow, dis_conc)
    Q_f = Q_i + dis_flow
    C_f = (Q_i*C_i + dis_flow*dis_conc) / Q_f
    return Q_f, C_f
end

# Crud concentration calculated using solution to diffeq, used AI to solve for solution
function CRUD(x, x0, C0, Q_flow)
    Ceq = no_mix(Q_flow)
    return Ceq + (C0 - Ceq) * exp(-k*(x - x0)/v)
end

# Initial concentration at river inflow
C_0 = Q_d

# Immediate mixing at x1 = 0
Q_1, C_1 = mix(Q, C_0, s1, s1_d)

# Concentration at x2 before mixing
C_2 = CRUD(x2,x1,C_1,Q_1)

# Mixing at x2
Q_3, C_3 = mix(Q_1, C_2, s2, s2_d)

println("Concentration at river inflow: ", round(C_0, digits = 6), " kg/m3")
println("Concentration after mixing at x1: ", round(C_1, digits = 6), " kg/m3")
println("Concentration at x2 before mixing: ", round(C_2, digits = 6), " kg/m3")
println("Concentration at x2 after mixing: ", round(C_3, digits = 6), " kg/m3")
println("")

# 2.3
reg_limit = 2.3 / 1000      #convert to kg/m3

if C_1 > reg_limit || C_2 > reg_limit || C_3 > reg_limit
    println("The system is not in compliance with the regulatory limit.")

```

```

else
    println("The system is in compliance with the regulatory limit.")
end

```

Problem 2.3

Determine if the system is in compliance with a regulatory limit of 2.3 kg/(1000m³). You can do this analytically or computationally.

Problem 3 (10 points)

In class, we discussed the ice-albedo feedback and its possible influence on melting the hypothesized “Snowball Earth”. In this problem, we’ll introduce a simple model of the Earth’s energy balance which includes this feedback and examine the stability of the climate.

This simple model of the energy balance (averaged over the entire planet) is:

$$\underbrace{C \frac{dT}{dt}}_{\text{change in heat}} = \underbrace{\frac{(1-\alpha)S}{4}}_{\text{incoming radiation}} - \underbrace{(A - BT)}_{\text{outgoing radiation}} + \underbrace{a \ln \left(\frac{[CO_2]}{[CO_2]_{PI}} \right)}_{\text{greenhouse effect}}, \quad (1)$$

where:

- T is the Earth’s global mean temperature (in °C);
- C is the heat capacity of the atmosphere and shallow ocean, taken to be 51 J/m²/°C;
- α is the planetary albedo, or the fraction of incoming radiation reflected by the Earth, which has a present-day value of approximately 0.3;
- S is the solar constant, or the amount of solar radiation received by the Earth averaged over area, which has a present-day value of 1368 W/m² but during the Neoproterozoic Era had a value of 1272 W/m² (this is divided by four because the Earth is a sphere but S is the radiation captured by a disc with radius equal to that of the Earth);
- A and B are coefficients from the linearization of outgoing radiation physics, and have corresponding values $B = -1.3$ W/m²/°C (estimated from a number of lines of evidence about the sensitivity of outgoing radiation to temperature; this is negative due to a sign convention about the direction of incoming vs. outgoing radiation) and $A = 221.2$ W/m² (estimated by assuming the pre-industrial temperature of 14°C was stable without anthropogenic greenhouse gas emissions).

We will ignore the greenhouse effect term as we are considering the Earth system well before humans were around.

Problem 3.1

Discretize the climate model (Equation 1) using forward Euler integration and a time step of $\Delta t = 1$ yr.

```
C = 51.0          # J / m^2 / C
A = 221.2         # W / m^2
B = -1.3          # W / m^2 / C (as given)
S_neoproera = 1272.0 # W / m^2 (Neoproterozoic)
alb_present = 0.3 # no units because its a ratio
S_present = 1361.0 # W / m^2 (present day)

#change in heat function of time
function derivative_function(T,S, alb)
    return (1 - alb) * S / 4.0 - (A - B * T) / C
end

# forward euler discretization
function forward_euler(T0, dt, t_final, S, alb)
    nsteps = Int(t_final / dt)
    T = zeros(nsteps)
    T[1] = T0
    for i in 1:nsteps-1
        dTdt = derivative_function(T[i], S, alb)
        T[i+1] = T[i] + dTdt * dt
    end
    return T
end
```

Problem 3.2

Rather than assuming a constant value for the albedo α , we will represent the ice-albedo feedback by letting α depend on T :

$$\alpha(T) = \begin{cases} \alpha_i & \text{if } T \leq -10^\circ\text{C} \\ \alpha_i + (\alpha_0 - \alpha_i)((T + 10)/20) & \text{if } -10^\circ\text{C} \leq T \leq 10^\circ\text{C} \\ \alpha_0 & \text{if } T \geq 10^\circ\text{C}. \end{cases}$$

Let $\alpha_i = 0.5$ and $\alpha_0 = 0.3$. Simulate the simple climate model using the Neoproterozoic value of S with temperature-varying albedo for initial values of T spanning $-60^\circ\text{C} \leq T_0 \leq 30^\circ\text{C}$ over a period of 200 years. Plot the temperature trajectories. How many equilibria are there and what are their stabilities?

```

using Plots

C = 51.0          # J / m^2 / C
A = 221.2         # W / m^2
B = -1.3          # W / m^2 / C
S_neoproera = 1272.0 # W / m^2
alb_i = 0.5
alb_0 = 0.3

# temperature-dependent albedo
function albedo(T)
    if T <= -10
        return alb_i
    elseif T >= 10
        return alb_0
    else
        return alb_i + (alb_0 - alb_i) * (T + 10) / 20
    end
end

# change in heat function of time but using the function albedo in terms of T
function derivative_function(T, S)
    return ((1 - albedo(T)) * S / 4.0 - (A - B * T)) / C
end

# forward euler discretization
function forward_euler(T0, dt, t_final, S)
    nsteps = Int(t_final / dt)
    T = zeros(nsteps)
    T[1] = T0
    for i in 1:nsteps-1
        dTdt = derivative_function(T[i], S)
        T[i+1] = T[i] + dTdt * dt
    end
    return T
end

# simulate different time steps
dt = 1.0 #timestep which is 1 year
t_final = 200.0 #final time in years
T0_vals = -60:10:30 # range of temperatures, with a temperature step of 10 degrees to test s
time = 0:dt:t_final-dt

```



```

plot()
for T0 in T0_vals
    T = forward_euler(T0, dt, t_final, S_neoproera)
    plot!(time, T, label="$T0 °C")
end
xlabel!("Time (years)")
ylabel!("Temperature (°C)")
title!("Problem 3.2")

```

There is only one equilibria, which is around -50C. After about 150 years, all temperatures converge to -50C. The equilibria is stable.

Problem 3.3

One might hypothesize that one cause for the transition from Snowball Earth (the stable frozen equilibrium from Problem 3.2) was an increasing amount of incoming solar radiation (as expressed by an increase in S). Examine this hypothesis using our simple model. Is this factor enough to cause the planet to warm to the pre-industrial temperature of 14°C?

```

function simulate_S_increase(T0, dt, t_final, S_values)
    results = Dict{Float64, Vector{Float64}}{ }
    time = collect(0:dt:t_final-dt)
    for S in S_values
        T = forward_euler(T0, dt, t_final, S)
        results[S] = T
    end
    return results, time
end

# test a range of solar constants
S_values = 1272:25:1725 # slowly increasing solar radiation by 25 W/m2
dt = 0.1
t_final = 200.0
T0 = -30.0 # start in cold temperature

results, time = simulate_S_increase(T0, dt, t_final, S_values)

# Ensure S values are plotted in order and not randomly
ordered_S = sort(collect(keys(results)))

```

```
plot()
for S in ordered_S
    plot!(time, results[S], label="S = $S")
end
xlabel!("Time (years)")
ylabel!("Temperature (°C)")
title!("Problem 3.3: Effect of Increasing Solar Constant on Climate")
```

This factor is enough to warm the planet to 14C. S values over 1647 appear to move towards a higher equilibria than previously found. S values of 1697 and over reach 14C in less than 200 years.

References

Stack Overflow for questions about julia code. ChatGPT and GitHub assistant for debugging help